

编译器设计专题实验报告

实验三：LR(0) 项目集规范族生成

张凌曜 2234214014 计算机 2306

1 实验目的

构建给定文法的 LR(0) 项目集规范族，计算每个状态的闭包和 Goto 转移，并判断文法是不是 LR(0) 文法。

2 实验原理

LR(0) 项目就是在产生式右部放个圆点 “.” 表示分析到哪了。比如 $E \rightarrow \cdot E + T$ 表示正在识别 $E + T$ 但还没读入东西。

闭包运算：如果项目集里有点在非终结符前面的项目 ($A \rightarrow \alpha \cdot B\beta$)，就把 B 的所有产生式加上点在开头加进去，反复做直到没有新的了。

Goto 运算：把项目集里点前有 X 的项目的点往右移一位，然后对结果再求闭包。

冲突判断：某个状态里同时有移进项目和归约项目就是移进-归约冲突，多个归约项目就是归约-归约冲突。有冲突就不是 LR(0) 文法。

3 实验过程

3.1 文法输入

程序读取产生式，用 $\rightarrow$ 表示推导， $|$ 分隔多个右部：

```
1 # 四则运算表达式文法
2 E -> E + T | T
3 T -> T * F | F
4 F -> ( E ) | id
```

程序自动加增广产生式 $E' \rightarrow E$ 。

3.2 实现要点

闭包用工作队列实现，避免重复处理。项目集用 `set` 存，天然去重。规范族的构建用 BFS，对每个状态尝试所有符号的 Goto，如果得到的项目集已存在就记录转移，否则加为新状态。

4 测试结果

4.1 四则运算文法

$E \rightarrow E + T | T, T \rightarrow T * F | F, F \rightarrow (E) | id$

程序输出 12 个状态、22 个转移函数。部分状态：

$I_0: E' \rightarrow \cdot E, E \rightarrow \cdot E + T, E \rightarrow \cdot T, T \rightarrow \cdot T * F, T \rightarrow \cdot F, F \rightarrow \cdot (E), F \rightarrow \cdot id$
 $I_1: E \rightarrow \cdot E + T, F \rightarrow (\cdot E), E \rightarrow \cdot T, T \rightarrow \cdot T * F, T \rightarrow \cdot F, F \rightarrow \cdot (E), F \rightarrow \cdot id$
 $I_2: E' \rightarrow E \cdot, E \rightarrow E \cdot + T$

I_2 里 $E' \rightarrow E \cdot$ 是归约， $E \rightarrow E \cdot + T$ 是移进，存在移进-归约冲突。 I_4 和 I_{10} 也有类似问题。所以这个文法不是 LR(0) 的。

请输入文法文件路径: ===== 原始文法 =====

$E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$

===== 增广文法 =====

(0) $E' \rightarrow E$
 (1) $E \rightarrow E + T$
 (2) $E \rightarrow T$
 (3) $T \rightarrow T * F$
 (4) $T \rightarrow F$
 (5) $F \rightarrow (E)$
 (6) $F \rightarrow id$

非终结符: {E, E', F, T}

终结符: {(,), *, +, id}

===== LR(0) 项目集规范族 =====

共 12 个状态

--- I0 ---

$E' \rightarrow \cdot E$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

--- I1 ---

$E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow (\cdot E)$
 $F \rightarrow \cdot id$

--- I2 ---

$E' \rightarrow E \cdot$
 $E \rightarrow E \cdot + T$

--- I3 ---

$T \rightarrow F \cdot$

--- I4 ---

$E \rightarrow T \cdot$
 $T \rightarrow T \cdot * F$

--- I5 ---

$F \rightarrow id \cdot$

--- I6 ---

$E \rightarrow E \cdot + T$
 $F \rightarrow (E \cdot)$

--- I7 ---

$E \rightarrow E + \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

--- I8 ---

$T \rightarrow T * \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

--- I9 ---

$F \rightarrow (E) \cdot$

--- I10 ---

$E \rightarrow E + T \cdot$
 $T \rightarrow T \cdot * F$

--- I11 ---

$T \rightarrow T * F \cdot$

```

===== 状态转移关系 =====
I0 --(--> I1
I0 --E--> I2
I0 --F--> I3
I0 --T--> I4
I0 --id--> I5
I1 --(--> I1
I1 --E--> I6
I1 --F--> I3
I1 --T--> I4
I1 --id--> I5
I2 --+--> I7
I4 --*--> I8
I6 --)--> I9
I6 --+--> I7
I7 --(--> I1
I7 --F--> I3
I7 --T--> I10
I7 --id--> I5
I8 --(--> I1
I8 --F--> I11
I8 --id--> I5
I10 --*--> I8
共 22 个转移函数

===== GOTO 表 =====
      (      )      *      +      E      F      T      id
I0      I1
I1      I1
I2
I3
I4
I5
I6      I9
I7      I1
I8      I1
I9
I10
I11

      I7
      I8
      I3      I10      I5
      I11      I5

===== LR(0) 冲突检测 =====
[冲突] 状态 I2 存在移进-归约冲突:
  移进项目:
    E -> E . + T
  归约项目:
    E' -> E .
[冲突] 状态 I4 存在移进-归约冲突:
  移进项目:
    T -> T . * F
  归约项目:
    E -> T .
[冲突] 状态 I10 存在移进-归约冲突:
  移进项目:
    T -> T . * F
  归约项目:
    E -> E + T .
该文法不是 LR(0) 文法 (存在冲突)
Press any key to continue . . .

```

4.2 简单文法

$$S \rightarrow aB, B \rightarrow bB|c$$

生成 6 个状态、7 个转移，没有冲突，是 LR(0) 文法。

```

===== 状态转移关系 =====
I0 --a--> I1
I1 --B--> I2
I1 --b--> I3
I1 --c--> I4
I3 --B--> I5
I3 --b--> I3
I3 --c--> I4
共 7 个转移函数

===== GOTO 表 =====
      B      a      b      c
I0
I1      I2          I3      I4
I2
I3      I5          I3      I4
I4
I5

===== LR(0) 冲突检测 =====
该文法是 LR(0) 文法 (无冲突)
Press any key to continue . . .

```

5 问答题：DFA 词法规则与报错分析

用第一个 DFA 作为词法规则扫描 `test.src`（内容为 `x = 42 + y`）时，程序报了大量错误。

原因：第一个 DFA 的转移函数设计得比较简单，只能识别很少的 Token 类型。遇到不在其字符集范围内的字符就会拒绝：字母 `x`、`y` 没有定义识别转移，所以报 `Invalid character`；赋值符 `=` 也不在 DFA 的转移表中；部分数字和运算符同样超出 DFA 的处理范围。最后一个 `Invalid DFA state 3` 说明 DFA 在识别某个运算符时进入了一个没有出边的死状态，转移不完整。

根本问题是第一个 DFA 的状态和转移覆盖不够全。扩展后的 DFA 加入了 ID（标识符）、NUM（数字）、OPERATOR（运算符）的识别，就能正确处理四则运算表达式了。这也说明词法分析器的 DFA 设计必须覆盖源语言的所有合法 Token。

6 实验总结

闭包运算的正确性很关键，Goto 之后一定要再求闭包，顺序不能反。四则运算文法虽然有 LR(0) 冲突，但不代表不能用——后面实验四用 SLR(1) 就能解决。

问答题部分让我更直观地理解了 DFA 设计对词法分析的影响：DFA 的转移表就是词法规则的形式化，漏掉任何一个 Token 类型都会导致合法代码被错误拒绝。

A 附录：代码仓库

本实验的全部源码、报告及 Web 演示平台已托管至 GitHub:

<https://github.com/yaossss123/compilation-web>